S. nithesh
192565036

# SIMATS ENGINEERING
## ASSESSMENT TOOL 2

**COURSE NAME:** **Artificial Intelligence** **COURSE CODE: CSA17**

---

**COURSE OUTCOME COVERED**

***CO1:*** Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively.*( BL3)*

*Assessment Tool Weightage: 50%*

---

**QUESTIONS: 2**

**Duration : 1 Hour**
**Total Marks:30**

Annexure A

**<u>Constraint-Based Problem Solving</u>**

---

***Code of Conduct:***

*I* S.nithesh *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
*I understand that any violation of academic integrity rules will result in disciplinary action.*

***Signature of the student:*** S. Nithesh

level-4

CO1 - AT2 - AI

S. mithesh

192565036

Q1. Back tracking search for Bus Route Assignment.

Problem Formulation

- Variables; B1, B2, B3 (Representing the route)
- Domains : {A, B, C} for all variables
- Ordering : let's process variables in Order : B1 → B2 → B3
- Ordering of Routes : A<B<C (Routes

constraints:

1. B1 ≠ C
2. B2 < B3 (i.e Route of B2 come Route of B3)
3. All variables must be distinct (B1 ≠ B2, B1 ≠ B3, B2 ≠ B3)
4. B3 ≠ A
5. Each Bus gets exactly one route

Steps:

1. Assign B1

   B1 = A → Try

   - Constraint check 1 (B1 ≠ C) : A ≠ C → Pass

2. Assign B2 (given B1 = A)

   Try B2 = B

   - constraint check 3 (B2 ≠ B1) : B ≠ A → Pass

   Try B2 = A:

   - constraint check 3 (B2 ≠ B1) : A = A → Fail (conflict)

3. Assign B3 (given B1 = A, B2 = B)

   - Try B3 = A
     - constraints check 4 (B3 = A) : A = A → Fail
   - Try B3 = B
     - constraints check 3 (B3 ≠ B1, B3 ≠ B2) : C ≠ A, C ≠ B → Pass
   -

constraint check 2 (B2 < B3): B < C (2 < 3) → Pass

Final valid schedule

B1 → Route A

B2 → Route B

B3 → Route C

Q2. uniform cost search (UCS) for autonomous Forklift

Grid & cost setup

- grid size : 5 × 5 with coordinates (x, y) where (0, 0) is Top-left and (4, 4) is Bottom-Right.
- start (S) : (0, 0)
- goal (G) : (2, 2)
- Shelving (obstacles) : (0, 1), (1, 1)
- Narrow Aisles (cost = 2) (1, 0) (2, 1)
- Normal Aisles (cost = 1) : All other accessible cells

Step-by step Node Explanation:

- Priority Queue (PQ) : Sorted by path cost g(n) ascending
- Visited list : keeps track of nodes already expanded

Initial state

- PQ : [((0, 0), g = 0, path = [(0, 0)])]

steps:

1. Expand (0, 0) [g = 0]
   - Neighbours of (0, 0)
     - Down (0, 1) : Obstacle → skip
     - Right (1, 0) : Narrow aisle (cost 2) → g = 0 + 2 = 2

- PQ [((1,0) ,g = 2)]
- Visited {(0,0)}

2. Expand (1,0) [g=2]
   - Neighbors of (1,0):
     - Left (0,0) : Already visited → skip
     - Down (1,1): Obstacle → Skip
     - Right (2,0): Normal Aisle (cost 1) → g = 2 + 1 = 3
   - PQ : [((2,0), g = 3)]
   - Vistied : {(0,0), (1,0)}

3. Expand (2,0) [g=3]
   - Neighbors of (2,0):
     - Left (1,0) : Already visted → skip
     - Right (3,0) : Normal aisle (cost 1) → g = 3+1 = 4
     - Down (2,1) : Narrow aisle (cost 2) → g : 3+2 = 5
   - PQ : [((3,0), g=4), ((2,1), g=5)]
   - Visited [(0,0), (1,0)]

4. Expand (3,0) [g=4]
   - Neighbors of (3,0):
     - Down (3,1) : Normal asile (cost 1) → g = 4+1 = 5
     - Right (4,0) : Normal aisle (cost 1) → g = 4+1 = 5
   - PQ : [((2,1), g=5), ((3,1), g = 5), ((4,0), g=5)]
   - Visited : { (0,0) (1,0) (2,0) (3,0)}